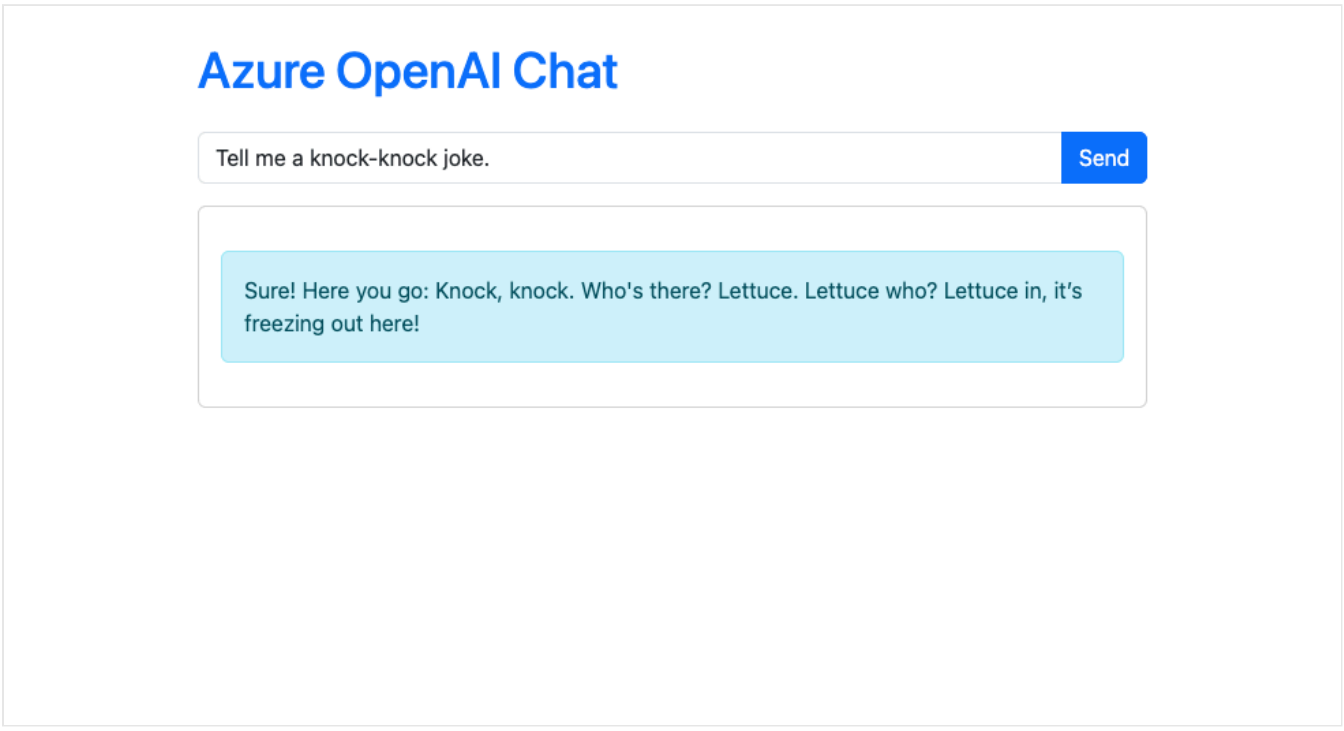
# Tutorial: Build a chatbot with Azure App Service and Azure OpenAI (Flask)

In this tutorial, you'll build an intelligent AI application by integrating Azure OpenAI with a Python web application and deploying it to Azure App Service. You'll create a Flask app that sends chat completion requests to a model in Azure OpenAI.




In this tutorial, you learn how to:

- ✓ Create an Azure OpenAI resource and deploy a language model.
- ✓ Build a Flask application that connects to Azure OpenAI.
- ✓ Deploy the application to Azure App Service.
- ✓ Implement passwordless secure authentication both in the development environment and in Azure.

## Prerequisites

- An Azure account with an active subscription
- A GitHub account for using GitHub Codespaces

## 1. Create an Azure OpenAI resource

In this section, you'll use GitHub Codespaces to create an Azure OpenAI resource with the Azure CLI.

1. Go to GitHub Codespaces and sign in with your GitHub account.

2. Find the **Blank** template by GitHub and select **Use this template** to create a new blank Codespace.

3. In the Codespace terminal, install the Azure CLI:

```bash
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

4. Sign in to your Azure account:

```azure-cli
az login
```

Follow the instructions in the terminal to authenticate.

5. Set environment variables for your resource group name, Azure OpenAI service name, and location:

```azure-cli
export RESOURCE_GROUP="<group-name>"
export OPENAI_SERVICE_NAME="<azure-openai-name>"
export APPSERVICE_NAME="<app-name>"
export LOCATION="eastus2"
```

> **Important**
>
> The region is critical as it's tied to the regional availability of the chosen model. Model availability and deployment type availability vary from region to region. This tutorial uses `gpt-4o-mini`, which is available in `eastus2` under the Standard deployment type. If you deploy to a different region, this model might not be available or might require a different tier. Before changing regions, consult the Model summary table and region availability to verify model support in your preferred region.

6. Create a resource group and an Azure OpenAI resource with a custom domain, then add a gpt-4o-mini model:

```azure-cli
```

```
# Resource group
az group create --name $RESOURCE_GROUP --location $LOCATION
# Azure OpenAI resource
az cognitiveservices account create \
  --name $OPENAI_SERVICE_NAME \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --custom-domain $OPENAI_SERVICE_NAME \
  --kind OpenAI \
  --sku s0
# gpt-4o-mini model
az cognitiveservices account deployment create \
  --name $OPENAI_SERVICE_NAME \
  --resource-group $RESOURCE_GROUP \
  --deployment-name gpt-4o-mini \
  --model-name gpt-4o-mini \
  --model-version 2024-07-18 \
  --model-format OpenAI \
  --sku-name Standard \
  --sku-capacity 1
# Cognitive Services OpenAI User role that lets the signed in Azure user to
read models from Azure OpenAI
az role assignment create \
  --assignee $(az ad signed-in-user show --query id -o tsv) \
  --role "Cognitive Services OpenAI User" \
  --scope /subscriptions/$(az account show --query id -o
tsv)/resourceGroups/$RESOURCE_GROUP/providers/Microsoft.CognitiveServices/accou
nts/$OPENAI_SERVICE_NAME
```

Now that you have an Azure OpenAI resource, you'll create a web application to interact with it.

# 2. Create and set up a Flask app

1. In your codespace terminal, create a virtual environment and install the PIP packages you need.

   Bash

   ```
   python3 -m venv .venv
   source .venv/bin/activate
   pip install flask openai azure.identity dotenv
   pip freeze > requirements.txt
   ```

2. In the workspace root, create an *app.py* and copy the following code into it, for a simple chat completion call with Azure OpenAI.

   Python

```
import os
from flask import Flask, render_template, request
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from openai import AzureOpenAI

app = Flask(__name__)

# Initialize the Azure OpenAI client with Microsoft Entra authentication
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://cognitiveservices.azure.com/.default"
)
client = AzureOpenAI(
    api_version="2024-10-21",
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    azure_ad_token_provider=token_provider,
)

@app.route('/', methods=['GET', 'POST'])
def index():
    response = None
    if request.method == 'POST': # Handle form submission
        user_message = request.form.get('message')
        if user_message:
            try:
                # Call the Azure OpenAI API with the user's message
                completion = client.chat.completions.create(
                    model="gpt-4o-mini",
                    messages=[{"role": "user", "content": user_message}]
                )
                ai_message = completion.choices[0].message.content
                response = ai_message
            except Exception as e:
                response = f"Error: {e}"
    return render_template('index.html', response=response)

if __name__ == '__main__':
    app.run()
```

3. Create a *templates* directory and an *index.html* file in it. Copy the following code into it for a simple chat interface:

HTML

```
<!doctype html>
<html>
<head>
    <title>Azure OpenAI Chat</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
</head>
```

```
<body>
    <main class="container py-4">
        <h1 class="mb-4 text-primary">Azure OpenAI Chat</h1>
        <form method="post" action="/" class="mb-3">
            <div class="input-group">
                <input type="text" name="message" class="form-control"
placeholder="Type your message..." required>
                <button type="submit" class="btn btn-primary">Send</button>
            </div>
        </form>
        <div class="card p-3">
            {% if response %}
                <div class="alert alert-info mt-3">{{ response }}</div>
            {% endif %}
        </div>
    </main>
</body>
</html>
```

4. In the terminal, retrieve your OpenAI endpoint:

```bash
az cognitiveservices account show \
  --name $OPENAI_SERVICE_NAME \
  --resource-group $RESOURCE_GROUP \
  --query properties.endpoint \
  --output tsv
```

5. Run the app by adding `AZURE_OPENAI_ENDPOINT` with its value from the CLI output:

```bash
AZURE_OPENAI_ENDPOINT=<output-from-previous-cli-command> flask run
```

6. Select **Open in browser** to launch the app in a new browser tab. Submit a question and see if you get a response message.

# 3. Deploy to Azure App Service and configure OpenAI connection

Now that your app works locally, let's deploy it to Azure App Service and set up a service connection to Azure OpenAI using managed identity.

1. First, deploy your app to Azure App Service using the Azure CLI command `az webapp up`. This command creates a new web app and deploys your code to it:

Azure CLI

```
az webapp up \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --name $APPSERVICE_NAME \
  --plan $APPSERVICE_NAME \
  --sku B1 \
  --os-type Linux \
  --track-status false
```

This command might take a few minutes to complete. It creates a new web app in the same resource group as your OpenAI resource.

2. After the app is deployed, create a service connection between your web app and the Azure OpenAI resource using managed identity:

Azure CLI

```
az webapp connection create cognitiveservices \
  --resource-group $RESOURCE_GROUP \
  --name $APPSERVICE_NAME \
  --target-resource-group $RESOURCE_GROUP \
  --account $OPENAI_SERVICE_NAME \
  --connection azure-openai \
  --system-identity
```

This command creates a connection between your web app and the Azure OpenAI resource by:

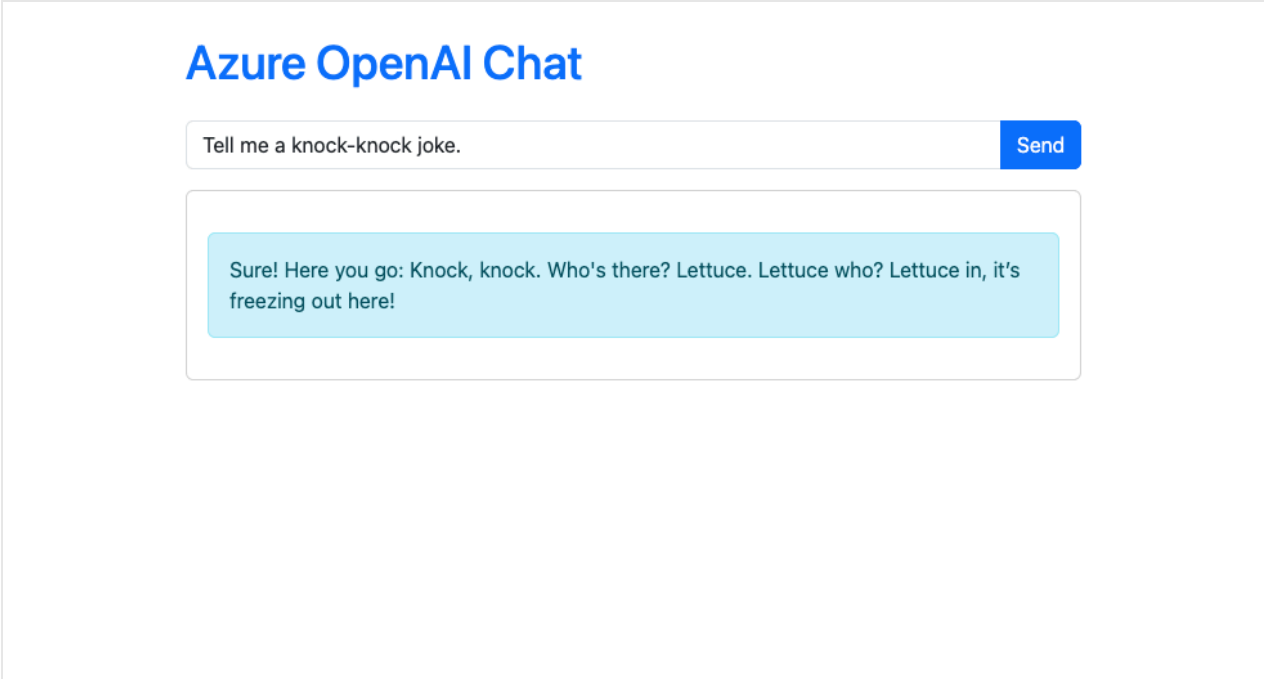
- Generating system-assigned managed identity for the web app.
- Adding the Cognitive Services OpenAI Contributor role to the managed identity for the Azure OpenAI resource.
- Adding the `AZURE_OPENAI_ENDPOINT` app setting to your web app.

3. Open the deployed web app in the browser. Find the URL of the deployed web app in the terminal output. Open your web browser and navigate to it.

Azure CLI

```
az webapp browse
```

4. Type a message in the textbox and select **Send**, and give the app a few seconds to reply with the message from Azure OpenAI.




Your app is now deployed and connected to Azure OpenAI with managed identity.

# Frequently asked questions

- What if I want to connect to OpenAI instead of Azure OpenAI?
- Can I connect to Azure OpenAI with an API key instead?
- How does DefaultAzureCredential work in this tutorial?

---

## What if I want to connect to OpenAI instead of Azure OpenAI?

To connect to OpenAI instead, use the following code:

Python

```python
from openai import OpenAI

client = OpenAI(
    api_key="<openai-api-key>"
)
```

For more information, see How to switch between OpenAI and Azure OpenAI endpoints with Python.

When working with connection secrets in App Service, you should use Key Vault references instead of storing secrets directly in your codebase. This ensures that sensitive information remains secure and is managed centrally.

---

## Can I connect to Azure OpenAI with an API key instead?

Yes, you can connect to Azure OpenAI using an API key instead of managed identity. This approach is supported by the Azure OpenAI SDKs and Semantic Kernel.

- For details on using API keys with Semantic Kernel: Semantic Kernel C# Quickstart.
- For details on using API keys with the Azure OpenAI client library: Quickstart: Get started using chat completions with Azure OpenAI Service.

When working with connection secrets in App Service, you should use Key Vault references instead of storing secrets directly in your codebase. This ensures that sensitive information remains secure and is managed centrally.

---

## How does DefaultAzureCredential work in this tutorial?

The `DefaultAzureCredential` simplifies authentication by automatically selecting the best available authentication method:

- **During local development**: After you run `az login`, it uses your local Azure CLI credentials.
- **When deployed to Azure App Service**: It uses the app's managed identity for secure, passwordless authentication.

This approach lets your code run securely and seamlessly in both local and cloud environments without modification.

# Next steps

- Tutorial: Build a Retrieval Augmented Generation with Azure OpenAI and Azure AI Search (FastAPI)
- Tutorial: Run chatbot in App Service with a Phi-4 sidecar extension (FastAPI)
- Create and deploy an Azure OpenAI Service resource
- Configure Azure App Service
- Enable managed identity for your app

---

Last updated on 05/19/2025